UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE LA INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN
ING. EDGAR RENÉ ORNELIS HOIL
ARQUITECTURA DE COMPUTADORAS II

“LOWMONITOR: SISTEMA DE DIAGNÓSTICO Y MONITOREO DE HARDWARE”

ALUMNOS:
FLAVIO WALDEMAR QUINTANILLA CÓRDOVA 1690 23 25004
YOZEF LEONIDAS EMMANUEL DUEÑAS SAGASTUME 0909-23-616
KEYLEE ZUGANY PÉREZ CHANG 0906-23-7745
SHIRLEY NOHEMÍ AGUILAR ESTRADA 0906-23-6452

GUATEMALA 07 DE NOVIEMBRE DE 2026

Introducción

LowMonitor es una aplicación de consola desarrollada en lenguaje ensamblador NASM para arquitectura x86-64 y ejecutada en Ubuntu sobre WSL2. Su objetivo es explorar características del procesador, permisos de entrada/salida, integridad de un bloque de memoria, contadores de rendimiento y diferencias de latencia entre tamaños de datos asociados a la jerarquía de memoria.

Objetivos

Objetivo general: implementar un diagnóstico experimental de hardware usando instrucciones x86-64 y servicios de Linux, documentando los resultados y las limitaciones del entorno.

Objetivos específicos: identificar el CPU con CPUID; comprobar la disponibilidad de permisos de E/S; validar patrones de memoria; estimar IPC y CPI mediante contadores; y comparar ciclos por acceso en distintos tamaños de datos.

Entorno y herramientas

Elemento	Implementación
Arquitectura	x86-64, formato ELF64
Sistema	Ubuntu en Windows Subsystem for Linux 2 (WSL2)
Ensamblador	NASM
Compilación	Makefile y enlazador ld
Interfaz	Menú de consola: módulos 1–5, ejecución conjunta y salida

El programa se compila con make rebuild y se ejecuta con make run. La opción 6 permite ejecutar consecutivamente los cinco módulos.

Desarrollo de los módulos

Identificación del procesador

El módulo emplea CPUID para leer el identificador del fabricante, el nombre comercial y las capacidades del procesador. La versión actualizada también obtiene la familia, el modelo numérico y el stepping a partir de los campos de CPUID (hoja 1). La detección de AVX verifica compatibilidad del procesador y habilitación del estado extendido mediante XGETBV; también se comprueban AVX2 y AVX-512F cuando corresponde.

Campo	Resultado observado
Fabricante	AuthenticAMD
Procesador	AMD Ryzen 7 7730U with Radeon Graphics
Arquitectura	x86-64
Familia / modelo / stepping	25 / 80 / 0
Extensiones mostradas	SSE, SSE2, AVX, AVX2

Monitor de puertos de E/S

Se presentan los puertos 60h (teclado), 61h (control del sistema) y 3F8h (serie COM1). El programa solicita permiso únicamente para el puerto 3F8h mediante la llamada al sistema ioperm (número 173 en x86-64), comprueba el resultado y revoca el permiso si se concede. Se contemplan EPERM, ENOSYS y otros errores.

Resultado observado en WSL2: ioperm devolvió ENOSYS (función no implementada). Por tanto, no se ejecutaron instrucciones físicas IN/OUT. Este módulo satisface la verificación de permisos, pero NO la parte de la rúbrica que exige operaciones reales IN/OUT de 8, 16 y 32 bits; esta limitación se declara expresamente.

Diagnóstico de memoria RAM

El módulo comprueba un búfer de 4096 bytes del espacio de memoria del proceso. Escribe y verifica tres patrones: 0xAA, 0x55 y secuencia incremental. Contabiliza bytes y bits incorrectos. En la prueba registrada se observaron cero errores. La prueba no equivale a un examen completo de la memoria RAM física.

Desarrollo de los módulos (continuación)

Análisis de rendimiento del CPU

Se utilizan contadores de rendimiento de Linux mediante perf_event_open para obtener ciclos de CPU e instrucciones retiradas durante una carga de trabajo de diez millones de iteraciones. Se calculan IPC = instrucciones / ciclos y CPI = ciclos / instrucciones.

Indicador	Ejemplo de ejecución registrado
Ciclos	30,152,263
Instrucciones	50,000,031
IPC	1.6582
CPI	0.6030

Las cifras corresponden a una ejecución previa y pueden variar según la carga del sistema. La medición utiliza contadores de rendimiento, no exclusivamente el contador de marca temporal RDTSC.

Jerarquía de memoria

Se aplica pointer chasing sobre índices de líneas de caché mezclados, para dificultar accesos secuenciales predecibles. Se comparan registros y conjuntos de datos de 16 KiB, 256 KiB, 8 MiB y 64 MiB. El tiempo se obtiene con RDTSC/RDTSCP; cada caso se repite cinco veces y se conserva el menor total observado, dividido entre la cantidad de accesos.

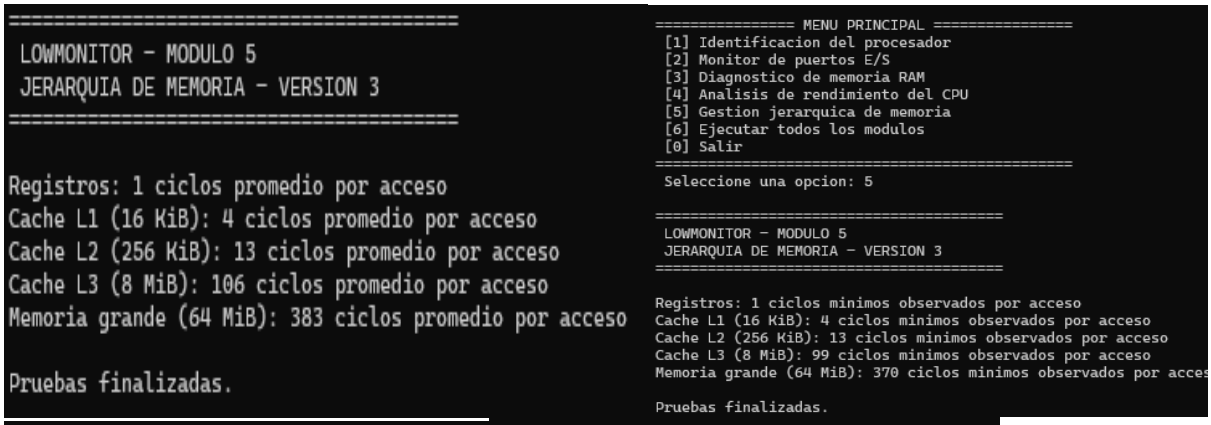
Prueba	Ejemplo: ciclos mínimos por acceso
Registros	1
16 KiB	4
256 KiB	13
8 MiB	99
64 MiB	370

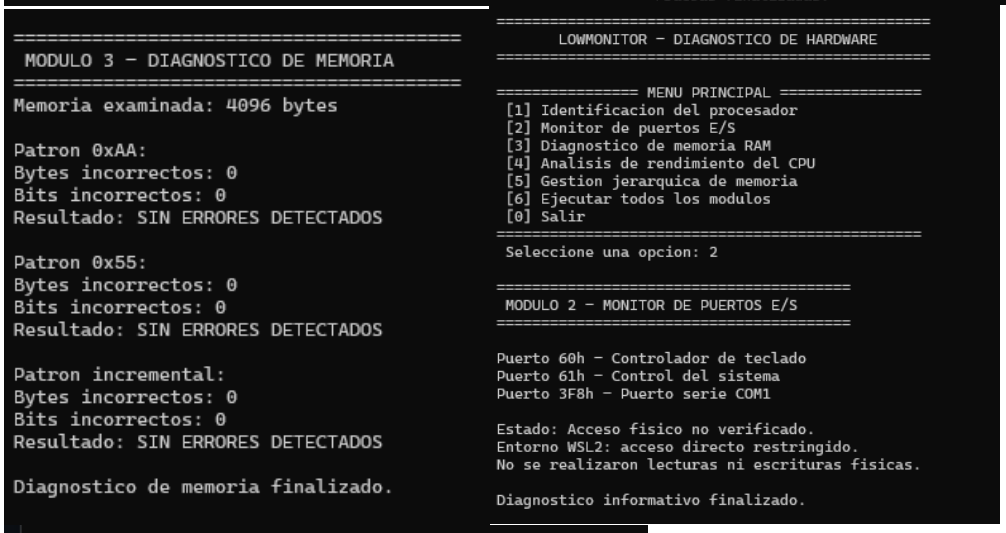
Los tamaños permiten observar tendencias compatibles con distintos niveles de la jerarquía, pero no identifican de manera absoluta la latencia aislada de L1, L2, L3 o RAM. Los resultados dependen del equipo y de la virtualización.

Pruebas y evidencias

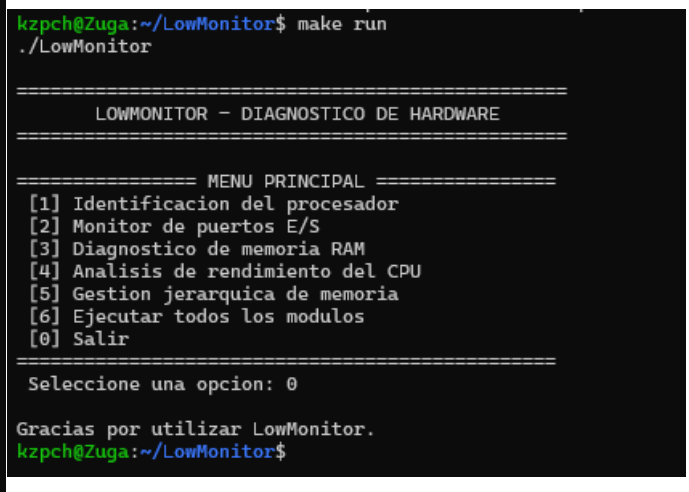
Se verificó el menú principal, la ejecución individual de los cinco módulos, la opción 6 y el retorno al menú.

Las seis capturas existentes se guardan en pruebas/evidencias/: 01_Menu_Principal.png, 02_Identificacion_CPU.png, 03_Puertos_ES.png, 04_Diagnostico_RAM.png, 05_Rendimiento_CPU.png y 06_Jerarquia_Memoria.png. La captura 02 fue actualizada después de incorporar familia, modelo y stepping.









Estructura, compilación y entrega

Archivos principales: src/main.asm, src/modulo1_cpu.asm, src/modulo2_puertos.asm, src/modulo3_memoria.asm, src/modulo4_rendimiento.asm y src/modulo5_jerarquia.asm. El directorio build/ contiene objetos de compilación; el ejecutable generado se llama LowMonitor. Se incluyen Makefile, README.md y pruebas/evidencias/.

Comando	Función
make rebuild	Reconstruye objetos y ejecutable ELF64
make run	Ejecuta ./LowMonitor
make clean	Elimina archivos generados de compilación

Limitaciones y alcance

La ejecución en WSL2 limita el acceso directo a puertos físicos; ioperm devolvió ENOSYS y no se realizaron operaciones IN/OUT. El diagnóstico de memoria cubre un búfer del proceso, no toda la RAM instalada. Las mediciones de rendimiento y latencia son experimentales y pueden cambiar entre ejecuciones. El módulo SIMD opcional no se implementó.

Conclusiones

LowMonitor integra cinco módulos de diagnóstico en ensamblador x86-64 y demuestra el uso de CPUID, comprobación de permisos del sistema operativo, patrones de memoria y mediciones de rendimiento. La actualización del primer módulo permite mostrar familia, modelo numérico y stepping. Los resultados de rendimiento evidencian diferencias entre cargas y tamaños de datos. La principal brecha frente a la rúbrica sigue siendo la ausencia de operaciones físicas IN/OUT en el módulo 2.

Referencias y material complementario

Intel Corporation. (s. f.). Intel® 64 and IA-32 Architectures Software Developer's Manual. Documentación de CPUID, RDTSC, RDTSCP e instrucciones IN/OUT.

The Linux man-pages project. (s. f.). ioperm(2), perf_event_open(2). Manual de interfaces del sistema Linux.

NASM. (s. f.). NASM documentation. Manual del ensamblador.

Adjuntos recomendados para la entrega: código fuente comentado, Makefile, ejecutable ELF64, capturas y enlace al repositorio GitHub/GitLab. Confirmar publicación del repositorio antes de entregar.